ECE 220 Computer Systems & Programming

Lecture 22 — Intro to C++: Iterators and Templates

November 12, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

Why `virtual` Needs a Table: `shape_choice.cpp`

```
int main() {
    Rectangle rec(3, 5);
    Triangle tri(4, 5);
    Shape *ptr;
    int which;
    cin >> which;
    if (which == 1) ptr = &rec;
    else            ptr = &tri;
    // 15 or 10?
    cout << ptr->area() << endl;
    return 0;
}
```

Lecture 21's `Shape`, `Rectangle` and `Triangle`, with `virtual double area()` in `Shape`. Input `1` prints 15, input `2` prints 10.

- `main` is compiled once, so `ptr->area()` is one call — `LDR`, `LDR`, `JSRR` — that must reach `Rectangle::area` on one run and `Triangle::area` on the next
- The compiler has only the declared type `Shape *`, so the call is `JSRR`: PC ← BaseR, with the two `LDR`s filling BaseR while the program runs
- An `if` chain on the type would need every derived class known when `main` compiles, and a library class can be derived from later
- So the object carries the address: one pointer per object, into a table shared by its class

```
R1 ← M[ptr]   the object's table
R2 ← M[R1]    entry 0, area
JSRR R2
```

getmin, Twice

getmin_overload.cpp:

```
int getmin(int a, int b) {
    return (a < b) ? a : b;
}

double getmin(double a, double b) {
    return (a < b) ? a : b;
}
```

Two functions, one body. A third type needs a third copy.

The overloads of Lecture 20's getmin.cpp.

getmin, Twice

getmin_overload.cpp:

```
int getmin(int a, int b) {
    return (a < b) ? a : b;
}

double getmin(double a, double b) {
    return (a < b) ? a : b;
}
```

Two functions, one body. A third type needs a third copy.

The overloads of Lecture 20's getmin.cpp.

getmin_t.cpp:

```
template <typename T>
T getmin(T a, T b) {
    return (a < b) ? a : b;
}

int main() {
    cout << getmin(3, 5) << endl;
    cout << getmin(2.5, 1.5) << endl;
    cout << getmin('x', 'b') << endl;
    cout << getmin<double>(3, 2.5) << endl;
    return 0;
}
```

It prints 3, 1.5, b, 2.5. One body serves int, double and char.

Today's Three Questions

A template is a function or a class with the type left as a parameter.

1. How does one `getmin` serve `int`, `double` and `char`, and what does the compiler build from it?
2. A `Person` keeps its name on the heap. What does `Person p2 = p1;` copy?
3. How does one loop walk an array, a linked list, a `vector` and a `list`?

The third of four C++ lectures. The library's `vector<T>` and `list<T>` are class templates; Lecture 24 uses both kinds of template to write a binary search tree for any type.

Function Template: `getmin<T>`

- `template <typename T>` makes `T` a **type parameter**: a name for a type, fixed at each call. `class` in place of `typename` means the same
- `getmin(3, 5)`: both arguments are `int`, so the compiler deduces `T = int`
- `getmin('x', 'b')`: `T = char`; it prints **b**
- `getmin<double>(3, 2.5)` names `T`; the 3 is converted to `3.0`. It prints **2.5**

Write `getmin(3, 2.5)` without `<double>`:

```
error: no matching function for call to 'getmin'
note: candidate template ignored: deduced
conflicting types for parameter 'T'
('int' vs. 'double')
```

The compiler writes one function for each type the program uses. `nm -C getmin_t.o` lists the functions in the object file:

```
T int getmin<int>(int, int)
T double getmin<double>(double, double)
T char getmin<char>(char, char)
```

- Writing such a function from the template is **instantiation**: `getmin<int>` is an instance of `getmin`
- A type the program never uses gets **no function**
- Each instance is compiled as if typed by hand, so it is as fast as the two overloads

When T Has No <: getmin_rect.cpp

```
class Rectangle {
    int width_, height_;
  public:
    Rectangle(int w, int h) {
        width_ = w; height_ = h;
    }
    int area() const { return width_ * height_; }
    // smaller means smaller area
    bool operator<(const Rectangle &r) const {
        return area() < r.area();
    }
};
```

```
int main() {
    Rectangle r1(3, 4), r2(2, 5);
    cout << getmin(r1, r2).area() << endl;
    return 0;
}
```

`getmin` is the template of the last slide. The program prints **10**.

A class can take a type parameter too. `mypair`, next.

Delete `operator<` from `Rectangle` and g++ reports:

```
error: invalid operands to binary expression
('Rectangle' and 'Rectangle')
return (a < b) ? a : b;
note: in instantiation of function template
specialization 'getmin<Rectangle>' requested
here
```

- The compiler checks the body once for each `T` used: here, with `T = Rectangle`, `a < b` has **no meaning**
- Every operation the body applies to `a` and `b` is a requirement on `T`: here `<` and **copying** (pass and return by value)
- The overloaded `operator<` supplies the `<`: `r2`, of area **10**, is the smaller

Class Template: mypair<T>

```
template <typename T>
class mypair {
    T a, b;
  public:
    mypair(T first, T second) {
        a = first; b = second;
    }
    T getmin();
};

template <typename T>
T mypair<T>::getmin() {
    T retval = (a < b) ? a : b;
    return retval;
}
```

```
int main() {
    mypair<int> myobject(100, 75);
    mypair<double> d(2.5, 1.5);
    cout << myobject.getmin() << endl;
    cout << d.getmin() << endl;
    cout << sizeof(myobject) << " ";
    cout << sizeof(d) << endl;
    return 0;
}
```

- `mypair<int>` is a class: the template with `T = int`. Its `a` and `b` are `int`s
- A member function defined outside the class repeats `template <typename T>` and names the class `mypair<T>::`
- It prints **75**, **1.5**, and the sizes **8** and **16**

The library's `vector<T>` and `list<T>` are class templates like `mypair<T>`, with a heap block inside.

Copying a Person: person_shallow.cpp

```
class Person {
    char *name_;
    int id_;
  public:
    Person(const char *name, int id);
    ~Person();
};
```

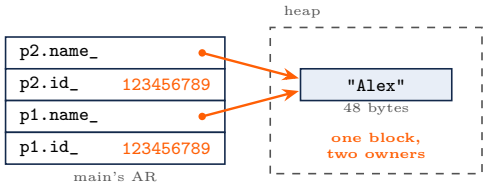
```
Person::Person(const char *name, int id) {
    name_ = new char[48];
    strcpy(name_, name);
    id_ = id;
}
Person::~Person() {
    cout << "delete[] " << (void *)name_ << endl;
    delete[] name_;
}
```

No copy constructor is declared: the compiler supplies one that copies every member. A **shallow copy**. Both destructors delete one block: a double free, undefined behavior.

```
    Person p1("Jamie", 123456789);
    Person p2 = p1;          // copy constructor
    p2.setName("Alex");
    p1.show();
    p2.show();
```

One run on a Mac:

```
Alex 123456789
Alex 123456789
delete[] 0x104c41a40
delete[] 0x104c41a40
Trace/BPT trap: 5
```



In-Class Problem: A Deep-Copy Copy Constructor

```
Person::Person(const Person &p) {
    name_ = new char[48];     // its own block
    strcpy(name_, p.name_);   // copy the chars
    id_ = p.id_;
}
```

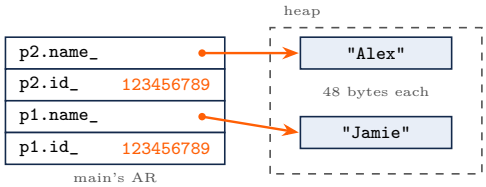
`main` is unchanged. Now it prints

```
Jamie 123456789
Alex 123456789
delete[] 0x101495920
delete[] 0x101495b00
```

`leaks --atExit -- ./person`: 0 leaks

A **deep copy** gives the new object a block of its own and copies the characters into it. When a member points to the heap, the copy constructor must copy what it points to.

Draw `main`'s AR and the heap just before `p1.show()`:



Two blocks, two addresses: each destructor deletes the block its own object allocated.

The Rule of Three: person3.cpp

```
    Person(const Person &p);                // 1
    Person &operator=(const Person &p);     // 2
    ~Person();                              // 3
```

```
Person &Person::operator=(const Person &p) {
    if (this != &p) {     // skip p1 = p1
        strcpy(name_, p.name_);  // own block
        id_ = p.id_;
    }
    return *this;
}
```

```
    Person p1("Jamie", 123456789);
    Person p2 = p1;     // copy constructor
    Person p3("Robin", 246813579);
    p3 = p1;            // copy assignment
    p2.setName("Alex");
    p3.setName("Sam");
```

Then p1, p2, p3 show **Jamie**, Alex, **Sam**, each with 123456789.

- `Person p2 = p1;` creates p2: it runs the **copy constructor**
- `p3 = p1;` copies into an object that already exists and owns a block: it runs `operator=`, the **copy assignment operator**
- Without it the compiler's `operator=` copies every member. `p3.name_ = p1.name_`: Robin's block is **leaked**, and Jamie's block is deleted **twice**
- `this != &p` skips `p1 = p1`, where `strcpy` would copy a block onto itself
- **Rule of three**: a class that needs its own destructor, copy constructor, or copy assignment operator needs all **three**

A `vector<Person>` copies each `Person` it stores. Containers, next.

vector<int>: an Array That Grows

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // a vector of four ints
    vector<int> intVector {1, 3, 5, 7};
    // two more at the end
    intVector.push_back(9);
    intVector.push_back(11);
    // print them, as with an array
    for (size_t i = 0; i < intVector.size(); i++)
        cout << intVector[i] << " ";
    cout << endl;
    return 0;
}
```

It prints 1 3 5 7 9 11

grow.cpp pushes 1 to 5 and prints v.size(), v.capacity() and &v[0]:

size	capacity	&v[0]
1	1	0x104e5da40
2	2	0x104e5da50
3	4	0x104e5da40
4	4	0x104e5da40
5	8	0x104e5da50

- A vector keeps its elements in one heap block, like an array; its destructor frees the block
- When size reaches capacity, push_back allocates a block **twice as large**, copies the elements, and frees the old block: what realloc does, done for you

A List Template of Our Own, and `friend`

```
template <typename T>
class Node {
    T data;
    Node<T> *next;
    Node(T d) { data = d; next = NULL; }
    friend class List<T>;   // List may use next
};
```

```
template <typename T>
class List {
    Node<T> *head;
  public:
    List() { head = NULL; }
    void add(T data) {      // insert at head
        Node<T> *ptr = new Node<T>(data);
        ptr->next = head;
        head = ptr;
    }
```

`List` also has a destructor that deletes every node, and `print`.

Lecture 24 builds a tree class template the same way.

- Every member of `Node` is private. `friend class List<T>;` lets `List<T>`'s member functions use them: `ptr->next`, `new Node<T>(data)`
- A **friend** (a class or a function) may use a class's private and protected members. The class grants it; friendship is one-way: `Node` may not use `List`'s **head**
- `main` adds 5, 2, 4 to `List<int>` and prints **4 2 5**
- `Node<int> n(3);` in `main`: **compile error**: calling a private constructor of class 'Node<int>'
- `leaks` reports **0 leaks**: the destructor deletes each node

Choosing Between `vector<T>` and `list<T>`

		vector	list
`size()`	number of elements	Y	Y
`push_back(x)`	add `x` at the end	Y	Y
`push_front(x)`	add `x` at the front	N	Y
`front()`, `back()`	first, last element	Y	Y
`insert(it, x)`	add `x` before position `it`	Y	Y
`erase(it)`	remove the element at it	Y	Y
`resize(n)`	make the size `n`	Y	Y
`v[i]`	element `i`, in one step	Y	N

`it` is an iterator, a position in the container: `v.begin() + 2` is element 2 of `v`.

vector<int>



list<int>



each node: prev, data, next

- `list<T>` is a doubly linked list: one node on the heap per element
- `insert` into the middle of a `vector` moves every later element; in a `list` it relinks two nodes
- A `list` has no `[]`: reaching element `i` takes `i` steps along `next`

How does a program walk a container whose elements are not in one block? Iterators, next.

One Loop Shape for Every Container: `loops.cpp`

walk	start	test	step	element
array `a[3]`	`int *p = a`	`p != a + 3`	`p++`	`*p`
linked list (C)	`node *q = &n1`	`q != NULL`	`q = q->next`	`q->data`
`vector<int> v`	`it = v.begin()`	`it != v.end()`	`++it`	`*it`
`list<int> l`	`it = l.begin()`	`it != l.end()`	`++it`	`*it`

```
vector<int> v {1, 2, 3};
for (vector<int>::iterator it = v.begin();
     it != v.end(); ++it)
    cout << *it << " ";
list<int> l {1, 2, 3};
for (auto it = l.begin(); it != l.end(); ++it)
    cout << *it << " ";
```

- An **iterator** is a variable, not a function: an object holding one address — a `T *` in a `vector`, a `Node *` in a `list` — with `*` and `++` overloaded
- It marks a position in the container. `begin()` is the first element; `end()` is one past the last, as `a + 3` is for `a`
- `*it` is the element; `++it` moves to the next. For a `vector` it adds **4 bytes** to an address; for a `list` it **follows next**
- `auto` lets the compiler write the type, `list<int>::iterator`

list<float>: unique and remove

```
void print(const list<float> &l) {
    for (auto it = l.begin(); it != l.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

int main() {
    list<float> l {2.1, 3.2, 4.5, 3.2, 5.6, 5.6};
    l.unique();        // adjacent duplicates
    print(l);
    l.remove(4.5);     // remove every 4.5
    print(l);
    l.unique();
    print(l);
    return 0;
}
```

It prints

```
2.1 3.2 4.5 3.2 5.6
2.1 3.2 3.2 5.6
2.1 3.2 5.6
```

- `unique()` removes a duplicate only when it is **next to** its copy
- `remove(4.5)` unlinks every node holding 4.5; the two 3.2s become **neighbors**
- `print` uses the same loop shape and works for any `list<float>`; it needs no size

In-Class Problem: One Template for Every Container

```
// print every element of c
template <typename C>
void print_all(const C &c) {
    for (auto it = c.begin(); it != c.end(); ++it)
        cout << *it << " ";
    cout << endl;
}
```

```
    vector<int> v {3, 1, 3, 2};
    list<char> l {'E', 'C', 'E', 'B'};
    print_all(v);
    print_all(l);
    cout << count_eq(v, 3) << " ";
    cout << count_eq(l, 'E') << endl;
```

```
// count the elements of c equal to x
template <typename C, typename T>
int count_eq(const C &c, T x) {
    int n = 0;
    for (auto it = c.begin(); it != c.end(); ++it)
        if (*it == x)
            n++;
    return n;
}
```

- `main` prints 3 1 3 2, E C E B, 2 2
- `C` needs `begin()`, `end()`, and an iterator with `!=`, `++`, `*`
- `print_all(a)` for `int a[3]`: **compile error**: member reference base type 'const int[3]' is not a structure or union

Summary & What's Next

Today

- **Function templates**: `template <typename T>` writes `getmin` once. The compiler deduces `T` at each call and writes one function per type used; the body's operations, such as `<`, are requirements on `T`
- **Class templates**: `mypair<int>` and `List<int>` are classes built from a template. `friend` opens private members to another class or function
- **Rule of three**: a member that points to the heap needs a deep-copy copy constructor, a copy assignment operator, and a destructor. The compiler's versions copy the address
- **vector and list**: a growing array and a doubly linked list, as class templates from the library
- **Iterators**: `begin()`, `end()`, `*it`, `++it`: the loop of a pointer over an array, for every container

Next lecture (Tuesday, November 17)

- Trees, traversal and search: a binary search tree in C. Lecture 24 writes it again as a class template, `bst<T>`